The Relay: a rate-limited cited-answer pipeline

1. What I built and why. Python/asyncio pipeline over the three pinned arXiv papers (1706.03762v7, 2410.00037v1, 2507.07955v2): **retrieve** (BM25 over page-aware pieces; a tool, no tokens) → **gate** (refuse with no model call if the best piece covers <35% of the question's terms) → **generate** (streamed, cites `[n]`, must say "the papers do not say" otherwise) → **verify** (every claim cites an existing passage; one regeneration with more passages if not). Model: `openai/gpt-oss-20b` on Groq's free tier (`llama-3.1-8b-instant` is not on my account). **The decision that shaped it most:** the binding limit is *tokens per minute* (8,000, about 10 answers/min), not requests, so one shared limiter owns it and requests that cannot start within **8 s** are turned away at once, rather than queued to a 20 s deadline. Cost: at 5× overload 78% of people get an instant "no" instead of a slow "yes". Measured gain: served requests' median wait fell from 16.0 s to 6.0 s at the same throughput (40 vs 39 answered).

2. The real service and my stand-in. Limits are in Table 1, found by reading `x-ratelimit-*` headers and by hitting limits on purpose (a 40-request burst gave exactly 30 ok then `RPM: Limit 30`; a naive 10-request burst of ~1.4k-token prompts gave `TPM: Limit 8000, Used 7433, Requested 1477`). The stand-in is an SSE server with the same headers and 429 bodies; time-to-first-token (lognormal, median 768 ms, σ 0.34), output length (mean 70 tokens) and refusal rate (19.8%) are fitted to 91 real calls; prompt size had no effect on latency (r = 0.02) so none is modelled. It is **least like the real thing** in the tail and under load: its p99 first token is 1.7 s against 2.75 s real (n = 23, so indicative), and its failure, hang and load-slowdown rates are guesses, because I never pushed the real service into overload. Its answers are canned text, so I only graded quality on real runs.

3. Results. Tables 1–3 (last page) are computed from `results.csv` by `relay/analysis/tables.py`; peak queue and offered rate come from `raw/scenarios.jsonl`. Real runs: steady, burst, naive burst, retrieval sweep. Stand-in runs: sustained overload, limit halved, daily budget gone (limits can't be exhausted on the real account). Numbers we could not have guessed: **(a)** median time from first token to last is **21 ms** (n = 101), so first token ≈ total time (ratio 0.97): a reasoning model hides its thinking and streaming buys nothing; **(b)** the daily allowance refills continuously (`reset-requests` = used × 86.4 s), not at midnight, and rejected requests did not count against it (883 → 854 after 30 ok + 10 rejected); **(c)** one answer is charged about 40 more tokens than the API reports (138 vs 96 on a 1-request probe); **(d)** 15 of 15 immediate retries failed. Quality, 23 questions I wrote (18 answerable, 5 not), chosen setting: 60.9% correct and cited, 4.3% wrong, 21.7% correctly refused, 13.0% wrongly refused; 773 tokens per answer (≈ $0.07 per 1,000 answers at list price, notional).

4. Where the time goes. Slowest real request (`sweep_A-0018`): 3,222 ms total = 0.2 ms waiting in my queue + 0 ms retrying + 3,193 ms inside the provider call before the first token + 29 ms streaming. So the delay came **before** the first token, and since a typical request of the same size (35 output tokens) is first-token ≈ 790 ms, about 2.4 s was provider-side. I can't split provider queueing from model thinking from the client side. Contrast `burst-0011`: 2,123 ms = 1,200 ms waiting in my queue + 922 ms working. A rate-limited request in the naive burst: 512 ms, all of it 4 rejected attempts.

5. When a limit is hit. *Per-minute:* sliding 60 s token and request windows at 90% headroom, FIFO. When committed tokens exceed 70% of the cap (or the queue builds) later arrivals are **degraded** (3→2 passages, ≤250 output tokens, no regeneration); if they still cannot start in 8 s they are **dropped**. Who is delayed: everyone admitted, by ≤8 s. Dropped: the tail of a burst. Weaker answer: 33 of 39 served at 5× overload. Why: tokens are the scarce resource, a smaller context is cheaper than refusing someone, and a quick no beats a slow one. The limiter also shrinks its belief after a 429 (×0.7) so an unannounced halving of both limits cost only 1 of 24 requests a provider rejection (66.7% served, 29.2% gave up waiting). *Per-day:* a daily-limit 429 (or remaining = 0) stops model calls entirely (section 7).

Retrieval: quality versus cost. Four settings, same 23 questions (Table 3). Piece size 120 / 3 pieces: 60.9% correct, 773 tokens. 200 / 6 / re-rank: 60.9%, 1,533 tokens. 200 / 4: 57.1% (n = 21; 2 requests lost to a bug I fixed afterwards). 200 / 5 / re-rank / stemming: 56.5%, 8.7% wrong. **I chose 120 / 3 / no re-rank:** equal best quality at half the tokens. Doubling context bought no question; q03, q11 and q14 were refused wrongly at all four settings (q15 failed at three), so the cause is the retriever, not the amount fetched. Near the limit I drop to 2 pieces; if the daily budget is gone I skip the model entirely.

How this library will mislead a naive retriever. (1) *Numbers live in table rows, headline numbers in the abstract:* "What BLEU does the **base** Transformer get on WMT14 En-De?" The abstract says 28.4 (that is the *big* model); the right 27.3 is a table row on p.8 that ranked 5th–7th, so every setting refused wrongly. (2) *Lexical mismatch:* "How many parameters does Helium have?" The text says "7B-parameter"; the answer piece ranked 22nd. "Mimi's" never matches "Mimi". Stemming fixed q15 but broke q04 and q16. (3) *One paper dominates and its appendix is noise:* Moshi is 51% of the text and its appendix holds sample conversations, so "What is the capital of France?" scores 5.6 BM25 there and passes a naive score threshold; "How does the Transformer compare to BERT?" fetches Moshi, the only paper that mentions BERT.

6. Retries. Up to 2 retries, only for 429, timeout and 5xx; wait = `retry-after` plus jitter; never past the deadline; a daily-limit 429 is not retried. **Where retrying made things worse:** the naive policy (no limiter, 3 immediate retries) on the real service: 5 of 10 requests failed after 4 attempts each within about 0.5 s, though `retry-after` said 7–8 s, so 20 rejected

calls for 5 requests. At 5× overload on the stand-in it made 2.58 retries per request and served 15.6% against 21.7% with the limiter.

7. What the person sees. At 3 pm with the budget gone, in a median of 4 ms and with no model call: "Today's model capacity is used up, so here are the most relevant passages instead of a written answer:" plus the top 3 passages with paper, version and page. Half of those (5 of 10) contained the answer. Known gap: I flag the budget as gone and never clear it, but the real allowance refills about every 86 s per request.

8. Unsure, left out, learned. *Least certain:* that the 8 s patience matches how long people really wait (I did not measure it) and that hidden reasoning tokens are about 40 per call (one data point); if either is wrong the limiter over- or under-admits, and the 429 feedback is my only protection. *Not built:* dense/embedding retrieval or an LLM re-ranker (it would spend the scarce tokens and add failure modes); priority classes and answer caching. Also not measured: the real tokens-per-day limit. *New to me:* in a sliding-window limit, a rejected request needs `Requested − (Limit − Used)` tokens to age out, here 1,477 − 567 = 910 tokens, which is the 6.8 s the server told me; retrying sooner cannot succeed.

Your feedback on Vaani. The page's strongest claim is latency ("sub-500 ms", "~450 ms p90"), but it never says what is timed (end of speech to first audio?), at which load, or why both figures appear. A text-only fetch showed the counters as 0 (likely animation; not checked in a browser). I would define the latency number on the page, because it is what a buyer will test.

How I worked with the assistant. Full raw session: `history/conversation.md` (my API key redacted). *Prompts that mattered:* "make a plan and set up the project layout"; pasting my Groq key so it could probe the real service; "use auto mode, complete it". *What it got wrong, caught by its own measurements:* it graded three correct answers wrong (the model wrote `3.5 days` with a non-breaking space and cited as `【1】`), which also made the pipeline regenerate and then throw away good answers; it first queued every request up to a 20 s deadline (median first token 16.9 s under overload). *Decision overruled:* none in this session; I accepted its design and measurements. *What is mine:* the framing and the go-ahead only. The code, the 23 questions, the 120 / 3 choice and the 8 s patience were written or proposed by the assistant, so I must be able to explain and change each of them live.

Required results tables

Table 1: limits found in the real service

Limit	Value you found	How you found it	Confidence
Requests per minute	30	Burst of 40 tiny concurrent requests: exactly 30 ok, 10 rejected; body says `requests per minute (RPM): Limit 30, Used 30`	high
Tokens per minute	8,000	`x-ratelimit-limit-tokens: 8000`; naive burst of 10 real requests: 5 rejected with `TPM: Limit 8000, Used 7433, Requested 1477`; remaining-tokens refills continuously	high
Requests per day	1,000	`x-ratelimit-limit-requests: 1000`, falls by 1 per successful call; `reset-requests` = used × 86.4 s (rolling refill). Never reached	medium
Tokens per day	not measured	In no header and never reached; I did not spend the quota to find it (about 130k tokens used in all)	low
Requests in flight at once	no separate limit found (40 concurrent accepted until RPM bit)	Burst of 40 and burst of 8 produced no concurrency error	low
What it returns when a limit is hit	HTTP 429, JSON message naming the limit, integer `retry-after` header (2 to 8 s seen)	Seen in the probe burst and the naive real burst	high
Time until it recovers	RPM about 4 s (2 s stated); TPM 6.8 to 8 s stated, depends on request size	RPM: polled once per 3 s after the burst. TPM: from `retry-after`, not independently timed	medium

Table 2a: latency, one row per load scenario

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
steady (= sweep_A)	real	23	264.5	5.2	790 / 1185 / 2753	817 / 1273 / 2797
burst_naive_real	real	10	1.0	623.9	742 / 829 / 839	510 / 900 / 918
burst	real	12	2.4	303.4	1201 / 2011 / 2041	1186 / 2046 / 2107
overload_q20	stand-in	180	192.0	60.3	16897 / 19546 / 19741	8 / 19124 / 20013
overload_naive	stand-in	180	179.0	60.3	864 / 3410 / 4837	13 / 1020 / 3085
burst_standin	stand-in	40	3.4	711.6	1588 / 2983 / 3144	1278 / 2663 / 3032
overload	stand-in	180	185.6	60.3	6730 / 8188 / 9402	8 / 7820 / 8670
limit_halved	stand-in	24	172.6	8.3	689 / 7992 / 8184	617 / 15773 / 18733
daily_exhausted	stand-in	30	174.0	10.3	947 / 1541 / 1743	828 / 1544 / 1734

Table 2b: outcomes, same rows

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers (USD, notional)
steady (= sweep_A)	100.0	0.0	0.0	0.00	1	773	0.072
burst_naive_real	50.0	50.0	0.0	1.50	0	1441	0.126
burst	83.3	0.0	16.7	0.00	8	684	0.065
overload_q20	22.2	0.6	77.2	0.01	20	607	0.057
overload_naive	15.6	84.4	0.0	2.58	0	791	0.073
burst_standin	22.5	0.0	77.5	0.00	35	706	0.067
overload	21.7	0.0	78.3	0.01	9	603	0.057
limit_halved	66.7	4.2	29.2	0.12	2	613	0.056
daily_exhausted	66.7	33.3	0.0	0.00	1	630	0.058

Table 3: retrieval settings

Setting (piece size, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
size120_k3_rr0	23	60.9	4.3	21.7	773	790 / 1185	817 / 1273
size200_k4_rr0	21	57.1	4.8	23.8	1309	916 / 1112	919 / 1115
size200_k5_rr1_stem	23	56.5	8.7	21.7	1263	825 / 1354	858 / 1356
size200_k6_rr1	23	60.9	4.3	21.7	1533	797 / 1549	826 / 1576

Notes. Real = Groq `openai/gpt-oss-20b`; stand-in = my server calibrated from real measurements. Percentiles: linear interpolation over the requests that have the value (p95/p99 of 23 requests are near the maximum). Total time includes requests that were turned away (a turned-away request takes ms, so total p50 can be tiny in overload). Rate-limited % counts requests that got no model answer because of a limit (provider 429 or the passages-only reply after the daily budget). 'Timed out or given up' includes requests my limiter refused after 8 s patience. Table 3 percentages do not sum to 100: the rest are answers refused wrongly (3 of 23 at every setting), and size200_k4 has 21 graded rows (2 requests lost to a bug fixed afterwards; run kept as `steady_v0`). Cost is notional list price ($0.075 / $0.30 per M tokens), the free tier costs nothing. Tokens per answer count API-reported tokens only. Scenarios `overload_q20` (queue to 20 s) and `overload_naive` are in results.csv for comparison with `overload`.